

Multi-Agent AI

Data Lakehouse Platform

for Betting Site Data Intelligence

■ Student Open-Source Edition

Every tool in this project is 100% free and open-source

Generated: June 17, 2026

Suitable for B.Tech / M.Tech / MCA Final Year Projects

Big Data

Machine Learning

RAG & LLMs

Multi-Agent AI

■ **Note for Students:** This report has been specifically updated to replace ALL paid or proprietary tools with freely available open-source alternatives. CI/CD pipeline steps have been removed as they require institutional infrastructure. Every framework, library, and platform listed here can be installed and run on a personal laptop or a free-tier cloud VM. GitHub links and installation notes are included throughout.

1. Abstract

*This document presents a comprehensive architectural blueprint and implementation guide for a student-friendly, fully open-source Multi-Agent AI platform that ingests, processes, and analyses payment-related data from multiple online betting platforms (Melbet, 10cric, 22xbet, 22crick, etc.). The system automates the entire lifecycle — from high-scale web scraping, real-time streaming, and distributed storage in a Data Lakehouse, to advanced machine learning, vector indexing, Retrieval-Augmented Generation (RAG), and a multi-agent AI orchestration layer. The outcome is a unified intelligence engine that transforms unstructured payment data into actionable insights. **Every single technology referenced in this report is open-source and freely available**, making this project ideal for B.Tech, M.Tech, and MCA final-year projects.*

2. Introduction

Modern enterprises face the challenge of extracting value from vast, heterogeneous data streams originating from disparate sources — public websites, APIs, system logs, and external databases. In the context of online betting platforms, payment method data (UPI IDs, bank account numbers, IFSC codes, crypto wallet addresses, e-wallet details) is frequently updated and distributed across dynamic web pages. This project establishes a robust, scalable, and intelligent pipeline to capture such data, unify it into a single source of truth, and apply cutting-edge AI to derive business intelligence. The final layer comprises a collaborative multi-agent AI framework and a RAG interface that empowers users with secure, high-fidelity, context-aware retrieval — all using tools that are entirely free to use and available on GitHub.

3. Problem Statement

- Enterprises are inundated with unstructured and semi-structured data from multiple online sources.
- Current infrastructure lacks safe and efficient ingestion of high-velocity, heterogeneous data streams.
- There is no unified storage that combines Big Data scalability with relational capabilities (ACID, time-travel).
- Payment method data on betting sites changes frequently; capturing and validating it in near-real-time is critical.
- Traditional ETL pipelines do not incorporate AI or autonomous orchestration — there is a gap between raw data and actionable knowledge.

4. Project Objectives

#1	Automated Data Extraction	Scalable web scraping and API crawling for multiple betting sites using open-source Scrapy + Playwright.
----	---------------------------	--

#2	Real-Time Data Ingestion	Fault-tolerant, high-throughput streaming via Apache Kafka (open-source).
#3	Distributed Big Data Processing	Optimised compute workflows using Apache Spark (PySpark) — free on any machine.
#4	Unified Analytics Storage	Data Lakehouse with ACID transactions using Apache Iceberg + MinIO (both open-source).
#5	Advanced Predictive Modelling	ML models using scikit-learn and PyTorch — fully free libraries.
#6	Semantic Vector Indexing	Dense vector representations using free Sentence-Transformers + FAISS.
#7	Contextual Knowledge Retrieval	RAG pipeline with open-source LLMs to reduce hallucinations.
#8	Domain Specialisation	Fine-tuning open-source LLMs (Llama 2 / FLAN-T5) on proprietary payment datasets.
#9	Autonomous Orchestration	Multi-agent AI framework using Python asyncio — zero cost.
#10	Student Deployment	Containerisation via Docker + Podman on local machines or free-tier cloud VMs.

5. System Architecture Overview

The architecture follows a layered approach, ensuring separation of concerns and horizontal scalability. Each layer is independently deployable, and all components have open-source alternatives that students can run locally using Docker Compose.

Data Sources Layer	External betting websites (Melbet, 10cric, 22xbet, 22crick) and their payment method pages — publicly accessible URLs.
Data Acquisition & Ingestion	Distributed Scrapy spiders with Playwright for JS-rendered pages, proxy rotation, and pushing data into Apache Kafka topics.
Stream Processing Layer	Apache Flink reads Kafka topics in real time, performs windowed aggregations, filtering, enrichment, and writes to the Bronze Lakehouse table.
Storage Layer (Data Lakehouse)	Apache Iceberg table format on MinIO (S3-compatible, open-source). Tables organised into Bronze → Silver → Gold zones.
Processing & Analytics Layer	Apache Spark (PySpark) handles batch ETL, feature engineering, and ML model training via MLlib and scikit-learn.
AI & Semantic Layer	Sentence-Transformers generate embeddings; FAISS indexes vectors. A RAG pipeline integrates an open-source LLM with the vector store.
Multi-Agent Orchestration	Specialised Python asyncio agents (Scraper, Validator, Analyzer, Anomaly Detector, RAG Handler, Report Generator) co-ordinated by an Orchestrator.
Deployment & Monitoring	Docker Compose for local dev; optional Kubernetes for scale. Monitoring via Prometheus + Grafana (both open-source).

6. Detailed Implementation Phases

Phase 1 — Data Acquisition & Web Scraping

■ Open-Source Tools: Scrapy (MIT), Playwright (Apache 2.0), BeautifulSoup (MIT), Rotating-Proxies (MIT)

- Set up a Scrapy project with separate spiders for each target site (Melbet, 10cric, 22xbet, 22crick).
- Configure dynamic content handling using Playwright (open-source, replaces paid Browserless) for JavaScript-rendered payment sections.
- Implement free proxy rotation using scrapy-rotating-proxies and user-agent rotation to avoid IP blocking.
- Define data models for each payment type: UPI, Bank Transfer, Cryptocurrency, E-wallets.
- Extract structured fields using XPath/CSS selectors and regex; capture raw HTML for audit.
- Schedule scraping jobs via Apache Airflow (open-source, free) to run at configurable intervals.
- Store raw JSON outputs temporarily, then push to Kafka for downstream processing.

Phase 2 — Real-Time Streaming Ingestion

■ Open-Source Tools: Apache Kafka (Apache 2.0), Apache Flink (Apache 2.0), Confluent Community Edition

- Deploy Apache Kafka (open-source) locally using Docker Compose — no paid Confluent license needed.
- Implement a Kafka producer in the scraping pipeline to publish each extracted record.
- Use Apache Flink to deserialize JSON messages, apply windowed aggregations, filter incomplete records, and write enriched data to the Bronze Lakehouse table.
- Ensure exactly-once semantics and fault tolerance using Flink's built-in checkpointing and Kafka's transactional producer.
- Optionally use Redpanda (free tier) as a Kafka-compatible alternative that is easier to run on laptops.

Phase 3 — Data Lakehouse Implementation

■ Open-Source Tools: Apache Iceberg (Apache 2.0), MinIO (AGPL-3.0 free), Apache Spark (Apache 2.0), Project Nessie (Apache 2.0)

- Set up MinIO (open-source, S3-compatible object store) locally — zero cost, runs in Docker.
- Configure Apache Iceberg as the table format with Project Nessie or a local Hive Metastore for catalog management (both open-source).
- Create three table layers: Bronze (raw, partitioned by date), Silver (cleaned, validated, enriched), Gold (aggregated analytics).
- Enable ACID transactions, schema evolution, and time travel on Iceberg tables — all built-in features, no paid add-ons.
- Implement PySpark jobs to transform Bronze → Silver → Gold using Spark SQL and DataFrames.

Phase 4 — Machine Learning & Analytics

■ Open-Source Tools: scikit-learn (BSD), PyTorch (BSD), MLflow (Apache 2.0), Pandas (BSD), NumPy (BSD)

- Prepare feature sets from the Silver table (UPI presence, bank account fields, crypto, amount ranges).
- Train a Random Forest classifier (scikit-learn) to predict payment category; use cross-validation and hyperparameter tuning.
- Train an Isolation Forest model (scikit-learn) to detect anomalies in transaction amounts or unusual payment detail combinations.
- Apply K-Means clustering on numeric features to group similar payment methods and discover patterns.
- Evaluate models using accuracy, precision, recall, F1, and silhouette scores.
- Save and version trained models using MLflow (open-source model registry) — free to self-host.

Phase 5 — Semantic Vector Indexing & Search

■ Open-Source Tools: Sentence-Transformers (Apache 2.0), FAISS (MIT), Qdrant (Apache 2.0, open-source alternative)

- Generate descriptive text for each payment record (e.g., 'Site: Melbet, Type: UPI, ID: xyz@paytm').
- Use Sentence-Transformers (e.g., all-MiniLM-L6-v2 — free on HuggingFace) to produce 384-dim embeddings.
- Build a FAISS index (IndexFlatL2 or IVFFlat) with all embeddings; store metadata alongside.
- Implement a query interface that accepts natural language, converts to embedding, and retrieves top-k similar records.
- Optionally use Qdrant (open-source, Docker-based) as a production-grade vector database alternative to FAISS.

Phase 6 — Retrieval-Augmented Generation (RAG) Pipeline

■ Open-Source Tools: LangChain (MIT), HuggingFace Transformers (Apache 2.0), FLAN-T5 / Llama 2 (free weights), Ollama (MIT)

- Select an open-source LLM: Google FLAN-T5-Large (free on HuggingFace) or Llama 2 7B (free weights via Meta).
- Use Ollama (open-source) to run LLMs locally on a laptop without a GPU — ideal for students.
- Wrap the LLM with LangChain's HuggingFacePipeline and create a RetrievalQA chain using the FAISS/Qdrant index as retriever.
- Craft a custom prompt template instructing the LLM to answer strictly from provided context, reducing hallucinations.
- Implement a post-processing step that verifies answers against retrieved context using term-overlap checking.
- Expose the RAG system via a FastAPI REST endpoint (open-source) for integration with Streamlit dashboards.

Phase 7 — Multi-Agent AI Framework

■ **Open-Source Tools:** Python asyncio (stdlib, free), Apache Airflow (Apache 2.0), CrewAI (MIT), AutoGen (MIT)

- Define agent roles: Scraper Manager, Data Validator, Payment Analyzer, Anomaly Detector, RAG Query Handler, Report Generator, Orchestrator.
- Implement agents using Python asyncio (zero cost, in stdlib) for concurrent, event-driven message processing.
- Optionally use CrewAI or Microsoft AutoGen (both open-source) for higher-level agent orchestration with less boilerplate.
- Define a standard message format: sender, receiver, type, payload, timestamp, correlation ID.
- Implement the Orchestrator to register agents, route messages, trigger pipeline phases, and handle timeouts.
- Enable agent collaboration: Scraper Manager → Data Validator → Payment Analyzer → Anomaly Detector → Report Generator.

Phase 8 — Deployment & Monitoring

■ **Open-Source Tools:** Docker (Apache 2.0), Docker Compose (Apache 2.0), Prometheus (Apache 2.0), Grafana (AGPL-3.0 free)

- Containerise all services using Docker (free) with separate images for each component.
 - Use Docker Compose for local multi-container orchestration — no Kubernetes required for student projects.
 - For production-scale optional deployment, use K3s (open-source lightweight Kubernetes) on a free-tier Oracle Cloud VM.
 - Monitor system health with Prometheus (metrics collection) and Grafana (dashboards) — both open-source and Docker-ready.
 - Configure alerts for critical issues (high error rate, no data for >1 hour, anomaly score above threshold).
 - Use Uptime Kuma (open-source) as a simple, self-hosted monitoring alternative to commercial solutions.
-

7. Open-Source Technology Stack

■ Every tool below is 100% free and open-source — no paid licenses required

Category	Open-Source Tools & Frameworks	License
Web Scraping	Scrapy, Playwright, BeautifulSoup, scrapy-rotating-proxies	MIT / Apache 2.0
Message Queue & Streaming	Apache Kafka (Docker), Apache Flink, Redpanda (alternative)	Apache 2.0
Storage	MinIO (local S3), Apache Iceberg, Project Nessie (catalog)	AGPL / Apache 2.0
Distributed Processing	Apache Spark (PySpark), Spark SQL, MLlib	Apache 2.0
Machine Learning	scikit-learn, Pandas, NumPy, PyTorch, MLflow	BSD / Apache 2.0
Vector Indexing	Sentence-Transformers (HuggingFace), FAISS, Qdrant	Apache 2.0 / MIT
RAG & LLM	HuggingFace Transformers, LangChain, FLAN-T5, Llama 2, Ollama	Apache 2.0 / MIT
Multi-Agent Framework	Python asyncio (stdlib), CrewAI, Microsoft AutoGen	MIT / Apache 2.0
Containerisation	Docker, Docker Compose, K3s (lightweight Kubernetes)	Apache 2.0
Monitoring	Prometheus, Grafana, Uptime Kuma	Apache 2.0 / AGPL
API & Dashboard	FastAPI, Streamlit, Gradio	MIT / Apache 2.0
Workflow Scheduling	Apache Airflow (self-hosted)	Apache 2.0

8. Enhancements to Stand Out

The following optional enhancements are all achievable with open-source tools and significantly elevate the academic and technical value of this project:

- 1. Real-Time Interactive Dashboard** — Build a Streamlit or Gradio dashboard (both open-source) showing live scraping status, payment trends, and anomaly alerts with WebSocket updates.
- 2. Graph Database Integration** — Use Neo4j Community Edition (free) to store relationships between sites, payment methods, and users. Enables network analysis of common UPI IDs across sites.
- 3. Automated PDF Report Generation** — Generate weekly/daily summary reports using ReportLab or WeasyPrint (both open-source) with charts, tables, and trend analysis.
- 4. Anomaly Detection with Deep Learning** — Replace Isolation Forest with a PyTorch autoencoder for better anomaly detection on high-dimensional payment feature vectors.
- 5. Blockchain Verification** — For crypto payments, use Web3.py (open-source) to verify wallet addresses and transaction history on public blockchains (Ethereum, BSC).
- 6. Advanced NLP Entity Extraction** — Use spaCy (MIT) or NLTK (Apache 2.0) to extract structured entities from raw HTML text, improving payment details accuracy.
- 7. Multi-lingual Support** — Use langdetect + Helsinki-NLP MarianMT (HuggingFace) to detect and translate multi-language content (Hindi, English, etc.) into a unified language.
- 8. Federated Learning** — Use PySyft (Apache 2.0) or Flower (Apache 2.0) to train ML models across different sites without centralising sensitive payment data.
- 9. API Gateway** — Expose RAG and analytics via a FastAPI gateway with OAuth2 authentication (free, built into FastAPI) for external integrations.
- 10. Open Dataset Publication** — Publish anonymised, aggregated payment method statistics as an open dataset on Kaggle or HuggingFace Datasets for community benefit.

9. Free Student Resources & GitHub Links

Apache Kafka (Docker)	github.com/apache/kafka hub.docker.com/_/kafka
Apache Flink	github.com/apache/flink flink.apache.org
Apache Spark / PySpark	github.com/apache/spark spark.apache.org
Apache Iceberg	github.com/apache/iceberg iceberg.apache.org
MinIO Object Storage	github.com/minio/minio min.io/docs
Sentence-Transformers	github.com/UKPLab/sentence-transformers huggingface.co/sentence-transformers
FAISS Vector Search	github.com/facebookresearch/faiss
LangChain	github.com/langchain-ai/langchain python.langchain.com
Llama 2 (Meta, free weights)	llama.meta.com huggingface.co/meta-llama
Ollama (run LLMs locally)	github.com/ollama/ollama ollama.ai
Scrapy Web Scraping	github.com/scrapy/scrapy scrapy.org
Playwright Browser Automation	github.com/microsoft/playwright-python
MLflow Model Registry	github.com/mlflow/mlflow mlflow.org
CrewAI Multi-Agent	github.com/joaomdmoura/crewAI
AutoGen (Microsoft)	github.com/microsoft/autogen
Qdrant Vector Database	github.com/qdrant/qdrant qdrant.tech
Project Nessie (Iceberg catalog)	github.com/projectnessie/nessie
Prometheus + Grafana	github.com/prometheus/prometheus github.com/grafana/grafana
FastAPI	github.com/tiangolo/fastapi fastapi.tiangolo.com
Streamlit	github.com/streamlit/streamlit streamlit.io
Apache Airflow	github.com/apache/airflow airflow.apache.org
K3s (lightweight Kubernetes)	github.com/k3s-io/k3s k3s.io

10. Ethical & Legal Considerations

Compliance

Scraping must respect robots.txt and Terms of Service. Use rate limiting and polite delays to avoid degrading target site performance.

Data Privacy

Payment details (UPI IDs, bank accounts) are sensitive. Anonymise or pseudonymise data. Store with AES-256 encryption at rest using open-source tools like age or GPG.

Purpose Limitation

Use this project for educational, analytical, or fraud-detection research only — never for fraudulent or malicious activities.

Transparency

Document all data sources and processing methods clearly in your project report. If deployed in production, obtain legal and compliance approval.

Academic Integrity

Cite all open-source libraries and their licenses in your project report. Respect the licenses of every tool used.

User Consent

If the data includes any personally identifiable information, ensure consent has been obtained or that use falls under legitimate academic interest.

11. Project Implementation Checklist

Use this checklist to track your project progress. Tick items off as you complete each phase.

Data Acquisition

- Scrapy spiders coded and tested for each site
- Playwright configured for JS-rendered pages
- Proxy and user-agent rotation implemented
- Data models finalised for all payment types
- Scraping scheduled with Apache Airflow

Streaming (Kafka + Flink)

- Kafka cluster deployed via Docker Compose
- Producers and consumers implemented
- Apache Flink streaming job developed and tested
- Dead-letter queue configured for failed records

Data Lakehouse (Iceberg + MinIO)

- MinIO deployed and buckets created
- Iceberg tables created for Bronze, Silver, Gold
- PySpark ETL jobs written and scheduled
- Time-travel and schema evolution tested

Machine Learning

- Feature engineering pipeline completed
- Random Forest classifier trained and evaluated
- Isolation Forest anomaly detector trained
- K-Means clustering applied and analysed
- Models versioned in MLflow

Vector Search & RAG

- Sentence-Transformer embeddings generated
- FAISS or Qdrant index built
- LLM (FLAN-T5 or Llama 2) downloaded and tested
- LangChain RetrievalQA chain integrated
- FastAPI endpoint tested end-to-end

Multi-Agent Framework

- Base agent class implemented

- All 7 specialised agents coded
- Orchestrator built and tested
- Inter-agent messaging verified

Deployment & Monitoring

- Docker images built for all components
- Docker Compose file written and tested
- Prometheus metrics configured
- Grafana dashboard created

Optional Enhancements

- Streamlit real-time dashboard
- Neo4j graph database integration
- Automated PDF report generation
- Multi-lingual support

12. Conclusion

This report outlines a comprehensive, production-grade architecture for a Multi-Agent AI Data Lakehouse Platform tailored to ingest and analyse payment data from online betting sites — built entirely on **free, open-source software** accessible to any student worldwide.

By combining modern Big Data technologies (Kafka, Flink, Iceberg, Spark) with advanced AI (Sentence-Transformers, FAISS, RAG, open-source LLMs) and autonomous agent-based orchestration (Python asyncio, CrewAI), the system transforms raw, messy web data into a clean, queryable knowledge base.

The step-by-step implementation guide, fully open-source technology stack, GitHub resource links, and enhancement suggestions provide a clear roadmap for any student to build a similar platform as a B.Tech / M.Tech / MCA final-year project. With careful attention to ethical and legal boundaries, this project demonstrates cutting-edge AI-driven data intelligence in a dynamic, real-world domain.

■ Built with 100% open-source tools — Free for all students — Share freely